

HPE CPP

Week 1 Technical Report

Title: In-Depth Analysis of QUIC Protocol, Wire-Level Behavior, and QoS Mechanisms

Submitted by: Shubham Kumar

1. Executive Summary

This report presents a structured technical analysis of the QUIC (Quick UDP Internet Connections) transport protocol as defined in RFC 9000 and RFC 9001. The study focuses on QUIC architecture, handshake mechanism, encryption levels, version negotiation, and connection ID management. A detailed comparison between QUIC and generic UDP at the wire level is also provided. Additionally, existing Quality of Service (QoS) and traffic policing mechanisms are examined to understand how encrypted transport protocols like QUIC interact with modern network control systems.

2. Introduction to QUIC

QUIC (Quick UDP Internet Connections) is a modern transport protocol that operates over UDP and is primarily used by HTTP/3 to provide faster and more secure web communication. Unlike traditional TCP-based communication, QUIC integrates TLS 1.3 encryption directly into the transport layer, significantly reducing connection establishment latency while improving security.

QUIC supports multiplexed streams within a single connection, eliminating head-of-line blocking that affects TCP-based protocols. This results in improved performance, particularly in lossy or high-latency network environments. Although QUIC encrypts most transport and application-layer information, it still relies on visible IP and UDP headers for routing.

3. QUIC Handshake Mechanism

The QUIC handshake integrates TLS 1.3 directly into the transport layer, allowing authentication and key establishment to complete efficiently within one round trip (1-RTT). The handshake must accomplish three primary objectives: establishing a connection, authenticating the server using certificates, and deriving shared encryption keys.

3.1 Encryption Levels

QUIC defines encryption levels that represent stages of key usage rather than additional round trips. These levels include: Initial (temporary keys derived from the Connection ID), Handshake (stronger temporary keys), and 1-RTT (final secure keys used for application data). These levels do not imply separate round trips; instead, they reflect progressive cryptographic transitions.

3.2 Handshake Flow

Step 1 – Client Initial: The client sends a TLS ClientHello containing supported cipher suites, key exchange information, client random, and transport parameters. Initial encryption keys are derived from the Connection ID and are not secret, but prevent plaintext exposure.

Step 2 – Server Response: The server replies within the same RTT using Initial encryption for ServerHello and transitions to Handshake encryption in the same flight to send Certificate, CertificateVerify, Finished, and transport parameters.

Step 3 – Client Processing: The client validates the certificate, verifies the signature, derives handshake and final 1-RTT keys, and sends its Finished message.

Step 4 – 1-RTT Communication: Initial and Handshake keys are discarded. Final secure 1-RTT keys are installed and application data transmission begins.

3.3 0-RTT Optimization

For returning clients, QUIC supports 0-RTT data transmission, allowing the client to send encrypted application data immediately with ClientHello. While this reduces latency, it carries replay risks and is not fully forward secure until handshake completion.

4. QUIC Connections and Connection IDs

In QUIC, connections are identified primarily using Connection IDs (CIDs) rather than just IP address and port. Each endpoint independently selects the CIDs that its peer will use, allowing packets to be correctly routed even if the client's IP address or port changes due to migration or NAT rebinding. Multiple CIDs may exist for a single connection to support migration, load balancing, and privacy. CIDs must not expose information that enables external correlation, and their lifecycle is managed using `NEW_CONNECTION_ID` and `RETIRE_CONNECTION_ID` frames, subject to limits such as `active_connection_id_limit`.

When a packet arrives, QUIC matches it to an existing connection using the Destination Connection ID (or IP/port if zero-length CIDs are used). If no matching connection is found, the endpoint may send a Stateless Reset so the peer can quickly detect that the connection no longer exists. Packets inconsistent with connection state (e.g., wrong version or invalid encryption) are discarded, and if partial state changes occur before detecting an error, the endpoint must either revert those changes or close the connection. Both clients and servers carefully handle version mismatches, 0-RTT buffering, and unsupported versions to maintain correctness and security.

In load-balanced deployments, connection continuity must be preserved even if client addresses change. Servers may route packets internally using CID-based mechanisms or instruct clients to migrate to a specific backend using the `preferred_address` transport parameter. If migration is not supported, servers can advertise `disable_active_migration`. Deployments must also prevent stateless reset oracles, ensuring attackers cannot infer whether a CID corresponds to an active connection. Overall, QUIC defines flexible transport capabilities rather than a fixed API, enabling

applications like HTTP/3 to manage connections, 0-RTT data, stream limits, flow control, handshake status, keep-alive (PING), and immediate closure securely and efficiently.

5. QUIC Version Negotiation

In QUIC, Version Negotiation (VN) is used when a client initiates a connection with a version the server does not support. In such cases, the server replies with a VN packet listing the versions it does support. However, this response is only sent if the client's first UDP datagram meets the required minimum size; otherwise, the server may silently discard it to prevent amplification attacks. Therefore, clients that support multiple QUIC versions must send their initial packet sized according to the largest minimum requirement among those versions to ensure the server can legally respond. A server must never send a VN packet in response to another VN packet, and it may limit the number of VN responses it sends for example, by ignoring unsupported 0-RTT packets and waiting for a valid Initial packet instead.

On the client side, careful handling of VN packets is essential for correctness and security. A client supporting only one QUIC version must abandon the connection attempt upon receiving a VN packet, unless it has already successfully processed another packet (indicating the version was accepted) or the VN packet incorrectly includes the client's originally chosen version, which must then be ignored. While the specification defines the structure and basic behaviour of VN packets, the complete negotiation algorithm particularly protections against downgrade attacks is left for future standards. Additionally, certain reserved version numbers (such as patterns like `0x?a?a?a`) are intentionally used in VN or test packets to ensure implementations properly ignore unknown versions, improving robustness and forward compatibility.

6. Wire-Level Comparison: QUIC vs Generic UDP

At the wire level, QUIC and generic UDP differ fundamentally in structure and functionality. UDP is a minimal transport protocol that provides only basic features: source and destination ports, length, and checksum. It is connectionless, offers no reliability, ordering, congestion control, or encryption, and leaves all higher-level functionality to the application. In contrast, QUIC runs over UDP but adds a rich framing structure inside UDP datagrams, including packet headers with Connection IDs, packet numbers, version fields, and encrypted payload frames. This allows QUIC to implement reliable delivery, congestion control, and multiplexed streams entirely in user space while still leveraging UDP for flexible deployment.

Another key wire-level difference is encryption and handshake behavior. UDP packets are sent in plaintext (except for optional application-layer encryption), whereas QUIC integrates TLS 1.3 directly into its handshake, encrypting almost all packet contents including most control information after the initial exchange. QUIC packets also carry explicit packet numbers for loss detection and acknowledgment frames for reliability, features absent in generic UDP.

Additionally, QUIC supports connection migration using Connection IDs, enabling continuity even when IP addresses or ports change, something UDP alone cannot handle.

Overall, while UDP simply transports datagrams without maintaining state, QUIC defines a fully stateful, secure, and congestion-controlled transport protocol at the wire level. It transforms UDP from a basic message carrier into a foundation for a modern, encrypted, multiplexed transport layer optimized for performance, security, and scalability.

7. Existing QoS and Traffic Policing Mechanisms

Quality of Service (QoS) mechanisms are designed to regulate network traffic so that bandwidth is allocated fairly and performance-sensitive applications receive appropriate priority. Traffic policing algorithms such as the Token Bucket and Leaky Bucket control the transmission rate by limiting how much data can be sent over time, while traffic shaping smooths bursty transmissions to maintain a steady flow and reduce congestion spikes. These mechanisms help ensure predictable latency, reduced packet loss, and better utilization of available bandwidth, especially in shared or high-traffic networks.

For traffic classification and prioritization, network devices often rely on DSCP (Differentiated Services Code Point) markings in the IP header, which allow packets to be categorized into different service classes (e.g., low-latency voice, best-effort data). Queue management techniques such as Random Early Detection (RED) and Controlled Delay (CoDel) proactively manage congestion by dropping or marking packets before buffers overflow, thereby preventing bufferbloat and reducing excessive delay. These approaches improve overall network stability and responsiveness under load.

With protocols like QUIC, most transport-layer headers are encrypted, limiting traditional deep packet inspection (DPI) methods that rely on visible TCP flags or payload inspection. However, QoS enforcement can still function effectively by analysing observable metadata such as source/destination IP addresses, UDP port numbers, flow duration, packet size patterns, and timing characteristics. Thus, even though QUIC enhances privacy and security, network operators can still implement traffic management policies based on flow-level behaviour rather than packet content.

8. Conclusion

QUIC represents a significant evolution in transport protocol design, combining transport setup and TLS encryption into a single efficient handshake while enabling connection migration, stream multiplexing, and congestion-aware secure communication. Compared to generic UDP, QUIC provides reliability, encryption, and state management at the wire level. Through analysis

of handshake stages, version negotiation, and CID lifecycle, this study demonstrates QUIC's robustness and suitability for modern web communication.